

ASSIGNMENT – 3

TCP Client–Server Applications for Checksum– and CRC–Based Error Detection

AIM

To develop two TCP client-server applications in Python for error detection over the network. Assignment 3a protects online banking transactions using the 1's Complement Checksum technique: the client computes the checksum of the data words and sends data plus checksum to a threaded bank server, which verifies integrity before processing. Assignment 3b protects video frames in a smart city traffic surveillance system using the Cyclic Redundancy Check (CRC) technique: the client computes the CRC remainder by XOR division with a generator polynomial, appends it to the frame, and a threaded monitoring server re-divides the received frame and accepts it only if the remainder is zero.

Both applications simulate transmission errors (error-free, single-bit, multiple-bit, and burst) so the effectiveness of each technique can be analyzed.

PART 3a – SERVER CODE (CHECKSUM) – checksum_server.py

```
import socket
import threading

# banking transaction verification server using 1's complement checksum

print_lock = threading.Lock()

def log(msg):
    # lock so two threads never write into the same line
    with print_lock:
        print(msg)

def verify(words, checksum):
    # 1's complement sum of all words + checksum should give all ones (0xFF)
    total = 0
    for w in words:
        total += int(w, 2)
    total += int(checksum, 2)
    while total > 0xFF:
        # wrap the carry around back into the sum
        total = (total & 0xFF) + (total >> 8)
    return total == 0xFF

def handle_client(conn, addr):
    log(f"Client Connected: {addr}")
```

```
try:
    msg = conn.recv(1024).decode().strip()
    words, checksum = msg.split("|")
    words = words.split(",")
    log(f"[Client {addr[1]] Received data words: {words}")
    log(f"[Client {addr[1]] Received checksum: {checksum}")
    if verify(words, checksum):
        reply = "Checksum Status : Valid\nTransaction Processed Successfully"
        log(f"[Client {addr[1]] Checksum verified, transaction accepted")
    else:
        reply = "Checksum Status : Corrupted\nTransaction Rejected"
        log(f"[Client {addr[1]] Checksum mismatch, transaction rejected")
    conn.sendall(reply.encode())
except Exception as e:
    log(f"[Client {addr[1]] Error: {e}")
finally:
    log(f"Client {addr} disconnected.")
    conn.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("127.0.0.1", 5004))
server.listen(5)
print("Banking Transaction Verification Server Started...")
print("Server listening on 127.0.0.1:5004")

while True:
    conn, addr = server.accept()
    # one thread per client so multiple branches are served at the same time
    t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
    t.start()
```

PART 3a - CLIENT CODE (CHECKSUM) - checksum_client.py

```
import socket

# banking client, computes 1's complement checksum and sends data + checksum

def checksum_1s(words):
    # sum all words, wrap the carry, then invert to get the checksum
    total = 0
    for w in words:
        total += int(w, 2)
```

```
while total > 0xFF:
    total = (total & 0xFF) + (total >> 8)
return format(total ^ 0xFF, "08b")

def to_words(text):
    # break a text message into 8-bit ascii words
    return [format(b, "08b") for b in text.encode()]

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5004))

msg = input("Enter data words (8-bit, comma separated) or text message: ")
err = input("Introduce error? (0 none, 1 single-bit, 2 multiple-bit): ")

if "," in msg:
    words = [w.strip() for w in msg.split(",")]
else:
    words = to_words(msg)
print("Data words:", words)

checksum = checksum_1s(words)
print("1's complement checksum:", checksum)

if err == "1":
    # single-bit error: flip one bit of word 2 (simulates a noisy line)
    words[1] = format(int(words[1], 2) ^ 0b00010000, "08b")
    print("Single-bit error introduced in word 2 ->", words[1])
elif err == "2":
    # multiple-bit error: flip one bit in every word
    words[0] = format(int(words[0], 2) ^ 0b00100000, "08b")
    words[1] = format(int(words[1], 2) ^ 0b00010000, "08b")
    words[2] = format(int(words[2], 2) ^ 0b00000010, "08b")
    print("Multiple-bit errors introduced ->", words)

client.sendall((",".join(words) + "|" + checksum).encode())
print("Data and checksum sent to server...")
reply = client.recv(1024).decode()
print("Server reply:")
print(reply)
client.close()
```

PART 3a – SERVER OUTPUT

Banking Transaction Verification Server Started...

```
Server listening on 127.0.0.1:5004
Client Connected: ('127.0.0.1', 62226)
[Client 62226] Received data words: ['10101010', '11001100', '11110000']
[Client 62226] Received checksum: 10010111
[Client 62226] Checksum verified, transaction accepted
Client ('127.0.0.1', 62226) disconnected.
Client Connected: ('127.0.0.1', 62227)
[Client 62227] Received data words: ['10101010', '11011100', '11110000']
[Client 62227] Received checksum: 10010111
[Client 62227] Checksum mismatch, transaction rejected
Client ('127.0.0.1', 62227) disconnected.
Client Connected: ('127.0.0.1', 62228)
[Client 62228] Received data words: ['10001010', '11011100', '11110010']
[Client 62228] Received checksum: 10010111
[Client 62228] Checksum mismatch, transaction rejected
Client ('127.0.0.1', 62228) disconnected.
Client Connected: ('127.0.0.1', 62229)
[Client 62229] Received data words: ['01010100', '01110010', '01100001', '01101110', '01110011', '01100001', '01100011', '01110100', '01101001', '01101111', '01101110', '00111010', '00100001', '01000001', '01000011', '01000011', '00110001', '00110000', '00110001', '00100000', '01000100', '01100101', '01110000', '01101111', '01110011', '01101001', '01110100', '00100000', '01010010', '01110011', '00101110', '00110101', '00110000', '00110000', '00110000']
[Client 62229] Received checksum: 10111011
[Client 62229] Checksum verified, transaction accepted
Client ('127.0.0.1', 62229) disconnected.
Client Connected: ('127.0.0.1', 62230)
[Client 62230] Received data words: ['01000001', '01000011', '01000011', '00110001', '00110000', '00110000', '00110001', '00100000', '01010100', '01011000', '01001110', '00110100', '00110100', '00111001', '00111001', '00111001', '00110110', '00100000', '01000001', '01101101', '01101111', '01110101', '01101110', '01110100', '00100000', '00110010', '00110101', '00110000', '00110000']
[Client 62230] Received checksum: 11001010
[Client 62230] Checksum verified, transaction accepted
Client ('127.0.0.1', 62230) disconnected.
```

PART 3a – TEST CASE 1 (NO ERROR)

```
Message type: 8-bit binary words
Data words: ['10101010', '11001100', '11110000']
1's complement checksum: 10010111
Data and checksum sent to server...
Server reply:
Checksum Status : Valid
Transaction Processed Successfully
```

PART 3a – TEST CASE 2 (SINGLE-BIT ERROR)

```
Message type: 8-bit binary words
Data words: ['10101010', '11001100', '11110000']
1's complement checksum: 10010111
Single-bit error introduced in word 2 -> 11011100
Data and checksum sent to server...
Server reply:
Checksum Status : Corrupted
Transaction Rejected
```

PART 3a – TEST CASE 3 (MULTIPLE-BIT ERROR)

```
Message type: 8-bit binary words
```

```
Data words: ['10101010', '11001100', '11110000']
1's complement checksum: 10010111
Multiple-bit errors introduced -> ['10001010', '11011100', '11110010']
Data and checksum sent to server...
Server reply:
Checksum Status : Corrupted
Transaction Rejected
```

PART 3a – TEST CASE 4 (TEXT MESSAGE)

```
Message type: text message
Data words: ['01010100', '01110010', '01000001', '01101110', '01110011', '01100001', '01100011', '01110100', '01101001', '01101111', '01101110', '00111010', '00100000', '01000001', '01000011', '01000011', '00110001', '00110000', '00110000', '00110001', '00100000', '01000100', '01100101', '01110000', '01101111', '01110011', '01101001', '01110100', '00100000', '01010010', '01110011', '00101110', '00110101', '00110000', '00110000', '00110000']
1's complement checksum: 10111011
Data and checksum sent to server...
Server reply:
Checksum Status : Valid
Transaction Processed Successfully
```

PART 3a – TEST CASE 5 (BANKING TRANSACTION)

```
Message type: text message
Data words: ['01000001', '01000011', '01000011', '00110001', '00110000', '00110000', '00110001', '00100000', '01010100', '01011000', '01001110', '00110100', '00110101', '00111000', '00111001', '00110110', '00100000', '01000001', '01101101', '01101111', '01110101', '01101110', '01110100', '00100000', '00110000']
1's complement checksum: 11001010
Data and checksum sent to server...
Server reply:
Checksum Status : Valid
Transaction Processed Successfully
```

PART 3a – TEST CASE RESULTS

Test Case	Data Sent	Checksum	Error Introduced	Server Result
1 – No error	10101010, 11001100, 11110000	10010111	None	Valid – Transaction Processed Successfully
2 – Single-bit error	10101010, 11011100, 11110000	10010111	One bit flipped in word 2	Corrupted – Transaction Rejected
3 – Multiple-bit error	10001010, 11011100, 11110010	10010111	One bit flipped in each word	Corrupted – Transaction Rejected
4 – Text message	"Transaction: ACC1001 Deposit Rs.5000"	10111011	None	Valid – Transaction Processed Successfully
5 – Banking app	"ACC1001 TXN45896 Amount 2500"	11001010	None	Valid – Transaction Processed Successfully

PART 3b – SERVER CODE (CRC) – crc_server.py

```
import socket
import threading

# traffic monitoring server using CRC verification over tcp

print_lock = threading.Lock()

def log(msg):
    # lock so two threads never write into the same line
    with print_lock:
        print(msg)

def crc_verify(frame, poly):
    # divide the received frame by the generator polynomial (xor long division)
    frame = list(frame)
    n = len(poly)
    for i in range(len(frame) - n + 1):
        if frame[i] == "1":
            for j in range(n):
                frame[i + j] = "1" if frame[i + j] != poly[j] else "0"
    # remainder is the last (n-1) bits, all zeros means the frame is clean
    return "".join(frame[-(n - 1):]).count("1") == 0

def handle_client(conn, addr):
    log(f"Client Connected: {addr}")
    try:
        msg = conn.recv(1024).decode().strip()
        frame, poly = msg.split("|")
        log(f"[Client {addr[1]}] Received frame: {frame}")
        log(f"[Client {addr[1]}] Generator polynomial: {poly}")
        if crc_verify(frame, poly):
            reply = "CRC Verification Successful\nNo Error Detected\nFrame Accepted"
            log(f"[Client {addr[1]}] CRC remainder zero, frame accepted")
        else:
            reply = "CRC Verification Failed\nError Detected\nFrame Rejected"
            log(f"[Client {addr[1]}] CRC remainder non-zero, frame rejected")
        conn.sendall(reply.encode())
    except Exception as e:
        log(f"[Client {addr[1]}] Error: {e}")
    finally:
        log(f"Client {addr} disconnected.")
        conn.close()
```

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server.bind(("127.0.0.1", 5005))
server.listen(5)
print("Traffic Monitoring CRC Server Started...")
print("Server listening on 127.0.0.1:5005")

while True:
    conn, addr = server.accept()
    # one thread per client so all cameras are served at the same time
    t = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
    t.start()
```

PART 3b - CLIENT CODE (CRC) - crc_client.py

```
import socket

# traffic camera client, computes CRC and appends it to the frame

def crc_remainder(data, poly):
    # xor long division of data (padded with zeros) by the generator polynomial
    frame = list(data + "0" * (len(poly) - 1))
    n = len(poly)
    for i in range(len(frame) - n + 1):
        if frame[i] == "1":
            for j in range(n):
                frame[i + j] = "1" if frame[i + j] != poly[j] else "0"
    return "".join(frame[-(n - 1):])

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5005))

frame = input("Enter data frame (binary): ")
poly = input("Enter generator polynomial (binary): ")
err = input("Introduce error? (0 none, 1 single-bit, 2 multiple-bit, 3 burst): ")

print("Data frame:", frame)
print("Generator polynomial:", poly)
rem = crc_remainder(frame, poly)
print("CRC remainder:", rem)
tx_frame = frame + rem
print("Transmitted frame (data + CRC):", tx_frame)
```

```
if err == "1":
    # single-bit error: flip one bit in the data part
    frame = format(int(frame, 2) ^ 0b1000, "0%db" % len(frame))
    print("Single-bit error introduced ->", frame)
elif err == "2":
    # multiple-bit error: flip two separated bits in the data part
    frame = format(int(frame, 2) ^ 0b10001000, "0%db" % len(frame))
    print("Multiple-bit errors introduced ->", frame)
elif err == "3":
    # burst error: flip a run of 4 consecutive bits in the data part
    frame = format(int(frame, 2) ^ 0b1111000, "0%db" % len(frame))
    print("Burst error introduced ->", frame)

client.sendall((frame + rem + "|" + poly).encode())
print("Frame and polynomial sent to server...")
reply = client.recv(1024).decode()
print("Server reply:")
print(reply)
client.close()
```

PART 3b – SERVER OUTPUT

```
Traffic Monitoring CRC Server Started...
Server listening on 127.0.0.1:5005
Client Connected: ('127.0.0.1', 62238)
[Client 62238] Received frame: 11010110111110
[Client 62238] Generator polynomial: 10011
[Client 62238] CRC remainder zero, frame accepted
Client ('127.0.0.1', 62238) disconnected.
Client Connected: ('127.0.0.1', 62239)
[Client 62239] Received frame: 11010100111110
[Client 62239] Generator polynomial: 10011
[Client 62239] CRC remainder non-zero, frame rejected
Client ('127.0.0.1', 62239) disconnected.
Client Connected: ('127.0.0.1', 62240)
[Client 62240] Received frame: 1001100010001
[Client 62240] Generator polynomial: 1101
[Client 62240] CRC remainder non-zero, frame rejected
Client ('127.0.0.1', 62240) disconnected.
Client Connected: ('127.0.0.1', 62241)
[Client 62241] Received frame: 1110000100100101
[Client 62241] Generator polynomial: 10011
[Client 62241] CRC remainder non-zero, frame rejected
Client ('127.0.0.1', 62241) disconnected.
Client Connected: ('127.0.0.1', 62242)
[Client 62242] Received frame: 100100001
```

```
[Client 62242] Generator polynomial: 1101
[Client 62242] CRC remainder zero, frame accepted
Client ('127.0.0.1', 62242) disconnected.
```

PART 3b – TEST CASE 1 (NO ERROR)

```
Data frame: 1101011011
Generator polynomial: 10011
CRC remainder: 1110
Transmitted frame (data + CRC): 11010110111110
Frame and polynomial sent to server...
Server reply:
CRC Verification Successful
No Error Detected
Frame Accepted
```

PART 3b – TEST CASE 2 (SINGLE-BIT ERROR)

```
Data frame: 1101011011
Generator polynomial: 10011
CRC remainder: 1110
Transmitted frame (data + CRC): 11010110111110
Single-bit error introduced -> 1101010011
Frame and polynomial sent to server...
Server reply:
CRC Verification Failed
Error Detected
Frame Rejected
```

PART 3b – TEST CASE 3 (MULTIPLE-BIT ERROR)

```
Data frame: 1011101010
Generator polynomial: 1101
CRC remainder: 001
Transmitted frame (data + CRC): 1011101010001
Multiple-bit errors introduced -> 1001100010
Frame and polynomial sent to server...
Server reply:
CRC Verification Failed
Error Detected
Frame Rejected
```

PART 3b – TEST CASE 4 (BURST ERROR)

```
Data frame: 111001101010
Generator polynomial: 10011
CRC remainder: 0101
Transmitted frame (data + CRC): 1110011010100101
Burst error introduced -> 111000010010
Frame and polynomial sent to server...
Server reply:
CRC Verification Failed
Error Detected
Frame Rejected
```

PART 3b – TEST CASE 5 (NO ERROR, SHORT FRAME)

```
Data frame: 100100
Generator polynomial: 1101
CRC remainder: 001
Transmitted frame (data + CRC): 100100001
Frame and polynomial sent to server...
Server reply:
CRC Verification Successful
No Error Detected
Frame Accepted
```

PART 3b – TEST CASE RESULTS

Test Case	Data Frame / Generator	CRC Remainder	Error Introduced	Server Result
1 – No error	1101011011 / 10011	1110	None	CRC remainder zero - Frame Accepted
2 – Single-bit error	1101011011 / 10011	1110	One bit flipped (1101010011)	CRC remainder non-zero - Frame Rejected
3 – Multiple-bit error	1011101010 / 1101	001	Two bits flipped (1001100010)	CRC remainder non-zero - Frame Rejected
4 – Burst error	111001101010 / 10011	0101	4-bit burst flipped (111000010010)	CRC remainder non-zero - Frame Rejected
5 – No error	100100 / 1101	001	None	CRC remainder zero - Frame Accepted

TASKS PERFORMED

Task	Details	Result
3a - Compute checksum	1's complement sum of the 8-bit words with carry wrap-around, then bitwise inversion	Checksums 10010111, 10111011 and 11001010 produced by the clients
3a - Error detection	Single-bit and multiple-bit errors flipped before verification	Both error patterns detected, transactions rejected
3a - Error-free traffic	Binary words, text message and banking transaction sent clean	All verified as valid and processed successfully
3b - Compute CRC	XOR long division of the data frame (zero-padded) by the generator polynomial	Remainders 1110 (10011), 001 and 0101 (1101) computed
3b - Error detection	Single-bit, multiple-bit and burst errors flipped before verification	All three patterns detected, frames rejected
3b - Error-free traffic	Full frame divides evenly by the generator polynomial	Remainder zero at server, frames accepted
Threaded servers	One thread per accepted connection on ports 5004 and 5005	All five clients per part were served concurrently

LEARNING OUTCOMES

- I computed the 1's complement checksum with carry wrap-around and bitwise inversion.
- I implemented CRC generation and verification using XOR long division in Python.
- I built threaded TCP servers that verify integrity before processing data.
- I detected single-bit, multiple-bit, and burst errors with both techniques.
- I concluded that CRC catches burst errors well and suits multimedia and real-time traffic data.
- I observed that a print lock is needed when multiple client threads log to the same console.